CE314/887 Lab 1
NLTK and Counting Words in Corpora

Annie Louis

20 October, 2016

1 IMPORTANT NOTE

At the end of this lab, your instructor will sign your work and give you some feedback. The exercise is designed to help you track your progress early on in the module. Do not worry if you do not finish all the questions in this lab exercise. Please work through the remaining parts after lab and make sure you understand the concepts. Tell your instructor how much progress you have made and difficulties if any. No formal grades or marks will be assigned for this lab work. However, it is **compulsory** for you to make sure you attend this lab and **get your work signed before you leave**. This exercise happens for the first lab only. Solutions will be posted on the module website after the lab.

Also, please help out your classmates who may be new to Python. You can discuss with others how to approach problems, but make sure you have a good understanding of the methods.

2 What this lab is about

This lab explores the concepts learnt in lectures last week. You will use the Natural Language Toolkit (NLTK) for your labs. This first lab introduces you to NLTK and takes you through the library's methods for accessing corpora and working with text. In particular, this lab focuses on computing word counts from a corpus.

3 Using NLTK

NLTK is a set of Python libraries for doing NLP. To get started, open a terminal in your machine (if a Linux machine) and type:

```
$ python
```

This command opens the Python interpreter. If you are working with a Windows OS, find the Python 3.5 IDLE GUI (from START menu).

Now you can start typing Python commands at the Python prompt >>>. For example, to add two numbers, type:

```
>>> 6 + 4
10
```

NLTK is already installed in the labs. On your own machine, you can download and install the latest version of NLTK from this link `http://nltk.org/`.

The next step is to import the NLTK libraries into the environment, and also download some of the corpora we will be using in this exercise. To carry out this step, type:

```
>>> import nltk
>>> nltk.download()
```

In the ensuing dialog box, choose *book* which downloads all the corpora used in the examples in the NLTK book[1]. Click download and once the download is finished, you can close the dialog box.

You can import the corpora by typing:

```
>>> from nltk.book import *
```

The list shows 9 texts or *corpora*, some of which are English novels you may recognize. *Wall Street Journal* is a corpus of articles, totaling 1 million words from the Wall Street Journal newspapers (primarily financial news).

To list the words in a particular text, you can use the `tokens` method. For example, the first 5 tokens in the Moby Dick corpus (text1) can be obtained by typing:

```
>>> text1.tokens[0:5]
[u'[', u'Moby', u'Dick', u'by', u'Herman']
```

To do the same with the Wall Street Journal corpus type:

```
>>> text7.tokens[0:5]
[u'Pierre', u'Vinken', u',', u'61', u'years']
```

Note that you can see the list of corpora again by typing:

```
>>> texts()
```

You can also count the number of occurrences of a particular word using the `count` method.

```
>>> text1.count("Moby")
84
```

4 Strings and regular expressions

Before working with large corpora, let's take a simple example sentence and look at the issues which we should keep in mind while processing strings.

Try the following example:

```
>>> mysent = ''The cat sat on the mat.''
>>> from nltk import word_tokenize
>>> mysent_tokens = word_tokenize(mysent)
>>> mysent_tokens
['The', 'cat', 'sat', 'on', 'the', 'mat', '.']
>>> len(mysent_tokens)
7
```

`word_tokenize` is an inbuilt method which tokenizes a natural language string. Note that the tokenization is not based entirely on splitting by spaces. `len(mysent_tokens)` returns the length of the list of tokens given as argument.

Sometimes you don't want to consider punctuations the same way as other word tokens (for example

[1] `http://www.nltk.org/book/`

while counting how many words are there). To remove punctuations, you can use regular expressions. A regular expression allows you to *match* a string against a *pattern*. For example, you can check if the entire string matches some start and end characters. ie. does the string start with 'a' or 'd' and end with a 'k'. Or you can find if a portion of the string matches some pattern. An example of this second case, would be finding if a string x is a substring of string y. There are some string patterns which cannot be specified by regular expressions. We will not go into the details here but you can read more about it.[2]

One common way to use regular expressions in Python is through the `re.search` method. The following snippet shows how to search if "lang" is a substring of "natural language processing":

```
>>> from nltk import re
>>> m = re.search("lang", "natural language processing")
>>> m
<_sre.SRE_Match object at 0x10beaa920>
>>> m.start()
8
>>> m.end()
12
```

The `re.search` method returns a *match* object if there is a successful match. Calling `start`, `end` methods on the *match* object returns the start and end index of the query string in the given string.

Check what happens here:

```
>>> n = re.search("nature", "natural language processing")
>>> n
```

While in the above cases, the pattern was a specific string value, typically regular expressions involve ranges and wildcard characters, but there are also inbuilt shortforms for frequently used expressions.

Now let's say we consider as words, only those tokens which are alpha-numeric ie. they only contain alphabets, numbers, or the hypen symbol. The regular expression you can use for this pattern is "\w".

```
>>> mysent_tokens_nopunct = [word for word in word_tokenize(mysent)
if re.search("\w", word)]
>>> mysent_tokens_nopunct
['The', 'cat', 'sat', 'on', 'the', 'mat']
>>> len(mysent_tokens_nopunct)
6
```

The method `set` creates a set. A set by definition will only have unique items. So every word appears only once, repetitions are ignored. Try:

```
>>> set(mysent_tokens_nopunct)
```

Does this set look satisfactory to you? (Hint: what size did you expect the set to have?)

Browse this list of string methods in Python and find the method to use to solve the problem. `https://docs.python.org/2/library/stdtypes.html#string-methods`. [Hint: it is easier to apply the method before creating the tokens from `mysent`.]

After you have created the new `mysent_tokens`, use `set` again to make sure you are happy with the result. By calling the `len` method on this set, you can find the number of unique words or *types*.

```
>>> len(set(mysent_tokens_nopunct))
5
```

Now we can apply these techniques to the larger corpora from NLTK.

```
>>> moby_dick_tokens = text1.tokens
```

[2] `https://docs.python.org/2/howto/regex.html`

```
>>> moby_dick_tokens_nopunct = [word.lower() for word in moby_dick_tokens
if re.search("\w", word)]
>>> moby_dick_tokens_nopunct[0:5]
[u'moby', u'dick', u'by', u'herman', u'melville']
```

5 Test your understanding

Based on what you have learned in the previous sections, answer the following questions.

**** Remove punctuations and convert to lower case before doing any counting ****

1. What is the number of tokens in Moby Dick _____
2. What is the number of types in Moby Dick? _____

Type-token ratio is a measure of how diverse the lexical items in a text are. It is defined as the number of types divided by the number of tokens. The higher this ratio, we consider the text as having greater diversity in words, also known as lexical diversity. In other words, texts with higher lexical diversity use a greater variety of words as opposed to using the same words repeatedly. To get an intuition, suppose two texts have the same number of tokens, then the one with greater types is more lexically diverse. By combining the methods for counting the types and tokens, can you find out the type-token ratio of Moby Dick? Repeat with the Wall Street Journal (WSJ) corpus.

3. Moby Dick type-token ratio = _____
4. WSJ type-token ratio is = _____
5. Which of the two has more lexical diversity (WSJ or Moby Dick)? _____
6. Can you think of a reason why this corpus has more diversity? _____

We also spoke in class about estimating probabilities from a corpus.

7. What is the Maximum Likelihood Estimate (MLE) probability of the word "whale" in Moby Dick?
$P_{\text{moby\_dick}}(\text{"whale"}) =$ _____

8. What is the MLE probability of "whale" in the WSJ?
$P_{\text{WSJ}}(\text{"whale"}) =$ _____